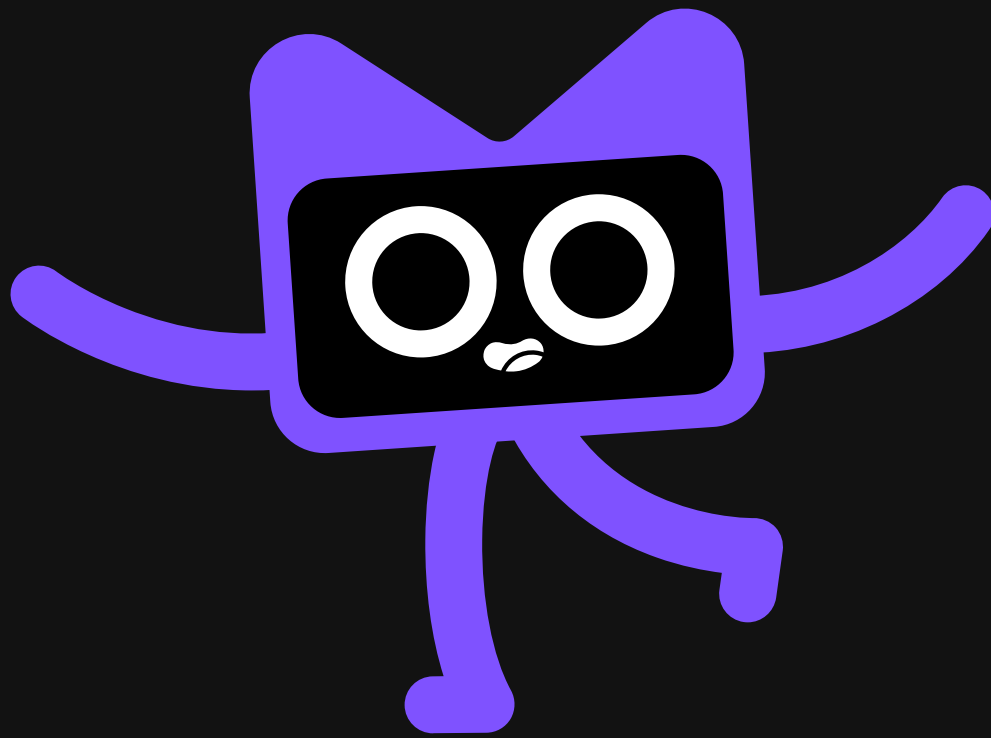


Why Spring Boot feels smaller in Kotlin

Frederik Pietzko



Why does Spring feel smaller in Kotlin?



Why does Spring feel smaller in Kotlin?

Kotlin features



Why does Spring feel smaller in Kotlin?

Kotlin features

- concise Syntax compare to Java
- Strict Nullability
- Smart Casts
- Extension Functions
- Default Arguments



Why does Spring feel smaller in Kotlin?

Kotlin features

for Spring

- concise Syntax compare to Java
- Strict Nullability
- Smart Casts
- Extension Functions
- Default Arguments



Why does Spring feel smaller in Kotlin?

Kotlin features

- concise Syntax compare to Java
- Strict Nullability
- Smart Casts
- Extension Functions
- Default Arguments

for Spring

- Kotlin Extensions for Spring
- Kotlin DSLs in Spring
- JSpecify
- Compiler Plugins
- Kotlin Ecosystem



Concise Syntax

```
public Pet getPet(Long id) {  
    return repository.findById(id)  
        .orElseThrow(() →  
            new IllegalArgumentException("Pet not found"))  
        );  
}
```



Concise Syntax

```
fun getPet(id: Long) = requireNotNull(repository.findByIdOrNull(id)) {  
    "Pet not found"  
}
```

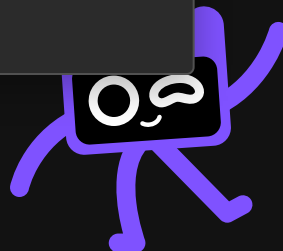
does the null check and throws an
IllegalArgumentException with provided message



Concise Syntax

```
@Service
class PetServiceImpl implements PetService {
    private final PetRepository repository;
    private final PetValidator validator;

    public PetService(PetRepository repository,
                      PetValidator validator) {
        this.repository = repository;
        this.validator = validator;
    }
}
```



Concise Syntax

```
@Service  
class PetServiceImpl(  
    private val repository: PetRepository,  
    private val validator: PetValidator,  
) : PetService
```

concise constructors & properties



Concise Syntax

```
@Service
class PetServiceImpl(
    private val repository: PetRepository,
    private val validator: PetValidator,
) : PetService
```

(
inheritance with :



Strict Nullability

```
public Visit scheduleVisit(Long petId, OffsetDateTime at) {  
    final var pet = petRepository.findById(petId).orElseNull();  
    final var petName = pet.getName();  
    return visitRepository.save(new Visit(petName, at))  
}
```

potentially throws NullPointerException



Strict Nullability

```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet: Pet? = petRepository.findByIdOrNull(petId)  
    val petName = pet.name • Only safe (?.) or non-null asserted (!!) calls are allowed on a nullable...  
    return visitRepository.save(Visit(petName, at))  
}
```



Strict Nullability

```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet: Pet? = petRepository.findByIdOrNull(petId) // Type: Pet?  
    if (pet == null) throw IllegalArgumentException(  
        "Pet with id: $petId not found!"  
    )  
    val petName = pet.name  
    return visitRepository.save(Visit(petName, at))  
}
```

pet is smart cast to Pet!



Strict Nullability

```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet: Pet = petRepository.findByIdOrNull(petId)  
        ?: throw IllegalArgumentException(  
            "Pet with id: $petId not found!"  
        )  
    val petName = pet.name  
    return visitRepository.save(Visit(petName, at))  
}
```

pet is smart cast to Pet!



Strict Nullability

```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet: Pet? = petRepository.findByIdOrNull(petId)  
    requireNotNull(pet) { "Pet with id: $petId not found!" }  
    val petName = pet.name  
    return visitRepository.save(Visit(petName, at))  
}
```

pet is smart cast to Pet!



Strict Nullability

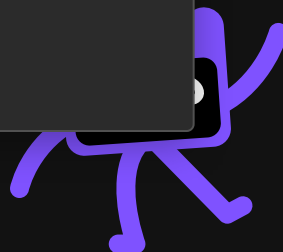
```
fun scheduleVisit(petId: Long, at: OffsetDateTime): Visit {  
    val pet = requireNotNull(petRepository.findByIdOrNull(petId)) {  
        "Pet with id: $petId not found!"  
    }  
    val petName = pet.name  
    return visitRepository.save(Visit(petName, at))  
}
```

pet is smart cast to Pet!



Strict Nullability

```
inline fun <T : Any> requireNotNull(  
    value: T?, lazyMessage: () → Any  
): T {  
    contract {  
        returns() implies (value ≠ null)  
    }  
  
    if (value == null) {  
        throw IllegalArgumentException(lazyMessage.toString())  
    } else {  
        return value  
    }  
}
```



Smart Casts

```
val pet = petRepository.findByIdOrNull(petId)
if (pet == null) {
    // pet has type Pet?
} else {
    // pet has type Pet!
}
```



Smart Casts

```
val stringOrInt: Any = if (Random.nextBoolean()) "string" else 1
if (stringOrInt is String) {
    // stringOrInt is smart cast to String
}
if(stringOrInt is Int) {
    // stringOrInt is smart cast to Int
}
// stingOrInt is Any
```



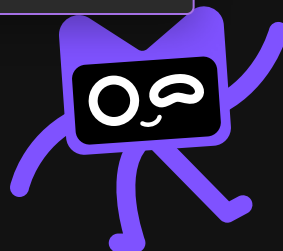
Smart Casts

```
val stringOrInt: Any = if (Random.nextBoolean()) "string" else 1
when(stringOrInt) {
  is String → // stringOrInt is smart cast to String
  is Int → // stringOrInt is smart cast to Int
  else → // stringOrInt is Any
}
```



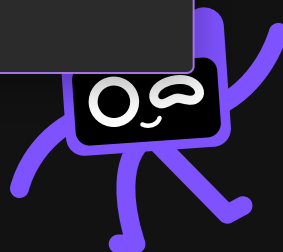
Smart Casts

```
sealed interface SchedulingResult {  
    data class Scheduled(val visit: Visit): SchedulingResult  
    data class SlotTaken(val takenBy: Visit): SchedulingResult  
    data object VetOnVacation: SchedulingResult  
}  
  
return when(val result = scheduleVisit(...)) {  
    is Scheduled → result.visit  
    is SlotTaken → error("Slot unavailable")  
    VetOnVacation → error("Vet on vacation")  
}
```



Smart Casts

```
sealed interface SchedulingResult {  
    data class Scheduled(val visit: Visit): SchedulingResult  
    data class SlotTaken(val takenBy: Visit): SchedulingResult  
    data object VetOnVacation: SchedulingResult  
    data object VetSick: SchedulingResult // newly added  
}  
  
return when(val result = scheduleVisit(...)) { • when must be exhaustive  
    is Scheduled → result.visit  
    is SlotTaken → error("Slot unavailable")  
    VetOnVacation → error("Vet on vacation")  
}
```



Smart Casts

```
sealed interface SchedulingResult {  
    data class Scheduled(val visit: Visit): SchedulingResult  
    data class SlotTaken(val takenBy: Visit): SchedulingResult  
    data object VetOnVacation: SchedulingResult  
    data object VetSick: SchedulingResult // newly added  
}
```

Data flow based exhaustiveness

```
if(result is Scheduled) return result.visit
```

```
return when(result) {  
    is SlotTaken → error("Slot unavailable")  
    VetOnVacation → error("Vet on vacation")  
    VetSick → error("Vet is sick")  
}
```


Extension Functions

```
val pet = repository.findByIdOrNull(petId)
```



Extension Functions

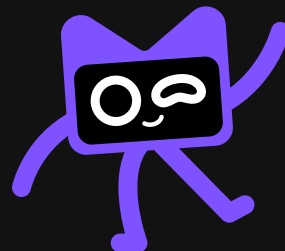
```
fun <T: Any, ID: Any> CrudRepository<T, ID>.findByIdOrNull(id: ID): T?  
    findById(id).orElse(null)
```



Extension Functions

```
@Entity
class Pet(
    @Id
    var id: Long? = null,
    var name: String
) {
    fun toDto() = PetDto(id, name)
}
```

polluted Domain Model



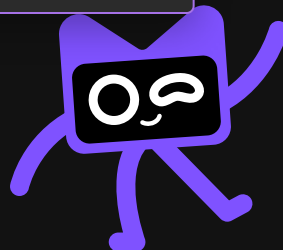
Extension Functions

```
package org.example.api

class PetController {
    @GetMapping
    fun getPet(id: Long): PetDto {
        val pet = requireNotNull(repository.findByIdOrNull(id))
        return pet.toDto()
    }

    private fun Pet.toDto() = PetDto(id, name)
}
```

Extension Function in Controller



Default Arguments

```
fun scheduleVisit(  
    petId: Long,  
    at: OffsetDateTime = OffsetDateTime.now() + Duration.ofDays(2)  
) : Visit {  
    // ...  
}
```



Default Arguments

```
fun scheduleVisit(  
    petId: Long,  
    at: OffsetDateTime = OffsetDateTime.now() + Duration.ofDays(2)  
) : Visit {  
    // ...  
}
```

How to use this from Java?



Default Arguments

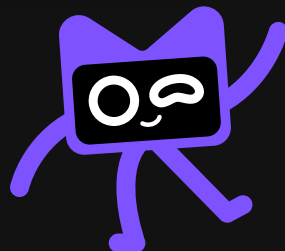
```
@JvmOverloads  
fun scheduleVisit(  
    petId: Long,  
    at: OffsetDateTime = OffsetDateTime.now() + Duration.ofDays(2)  
) : Visit {  
    // ...  
}
```

**Generates
Overloads
for
Java Callers**



Default Arguments

```
public Visit scheduleVisit(Long petId, OffsetDateTime at) {  
    // ...  
}  
  
public Visit scheduleVisit(Long petId) {  
    return scheduleVisit(  
        petId,  
        OffsetDateTime.now().plus(Duration.ofDays(2))  
    );  
}
```



Kotlin specific Extension Functions in Spring

```
val pet = petRepository.findByIdOrNull(id)
```



Kotlin specific Extension Functions in Spring

```
val pet = petRepository.findByIdOrNull(id)  
  
val pet = restTemplate.getForEntity<Pet>("/")
```



Kotlin specific Extension Functions in Spring

```
val pet = petRepository.findByIdOrNull(id)

val pet = restTemplate.getForEntity<Pet>("/")

val pet = jdbcTemplate.queryForObject<Pet>(...)
```



Kotlin specific Extension Functions in Spring

```
inline fun <reified T: Any> RestOperations.getForEntity(  
    url: String,  
    vararg uriVariables: Any?  
) : ResponseEntity<T> =  
    getForEntity(url, T::class.java, *uriVariables)
```

allows access to class of generic

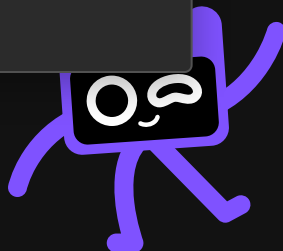


JSpecify

- Standard for nullability Annotations
- by Google, JetBrains, Meta, Microsoft, Oracle, Broadcom, ...
- Supported by IntelliJ IDEA, NullAway, Checker Framework
- and Kotlin!
- Spring Framework 7 & Spring Boot 4 completely annotated => seamless Kotlin interop

Kotlin DSLs in Spring

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) {
    return http
        .cors(cors → cors.disable())
        .csrf(csrf → csrf.disable())
        .authorizeHttpRequests(auth → auth
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().permitAll())
        .formLogin(Customizer.withDefaults())
        .httpBasic(Customizer.withDefaults())
        .build();
}
```



Kotlin DSLs in Spring

```
@Bean
fun filterChain(http: HttpSecurity): SecurityFilterChain {
    http {
        cors { disable() }
        csrf { disable() }
        authorizeHttpRequests {
            authorize("/admin/**", hasRole("ADMIN"))
            authorize(anyRequest, authenticated)
        }
        formLogin { }
        httpBasic { }
    }
    return http.build()
}
```

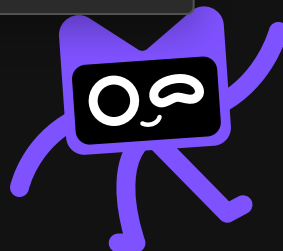
Kotlin DSLs in Spring

```
@Test
public void testSomething() throws Exception {
    mockMvc.perform(
        MockMvcRequestBuilders.get("/pet/{id}", 1)
            .accept(MediaType.APPLICATION_JSON)
    )
    .andExpect(MockMvcResultMatchers.status().isOk())
    .andExpect(MockMvcResultMatchers.jsonPath("$.id").value(1));
}
```



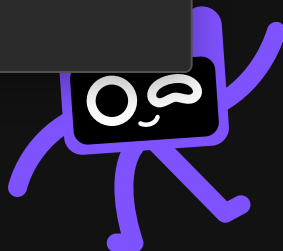
Kotlin DSLs in Spring

```
@Test
fun `test something`() {
    mockMvc.get("/pet/{id}", 1) {
        accept = APPLICATION_JSON
    }.andExpect {
        status { isOk() }
        jsonPath("$.id") {
            value(1)
        }
    }
}
```



Kotlin DSLs in Spring

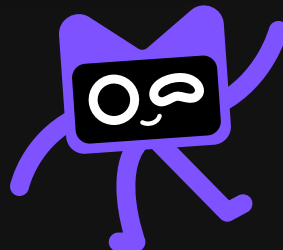
```
class Registrar implements BeanRegistrar {  
    @Override  
    public void register(BeanRegistry registry, Environment env) {  
        registry.registerBean(PetService.class);  
        if (env.matchesProfiles("dev")) {  
            registry.registerBean(TestingController.class);  
        }  
    }  
}  
  
@Configuration  
@Import(Registrar.class)  
class Config {}
```



Kotlin DSLs in Spring

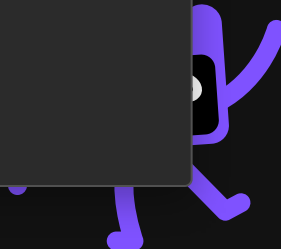
```
class Registrar : BeanRegistrarDsl({  
    registerBean<PetService>()  
    if (env.matchesProfiles("dev")) {  
        registerBean<TestingController>()  
    }  
})
```

```
@Configuration  
@Import(Registrar::class)  
class Config
```



Kotlin DSLs in Spring

```
class Registrar implements BeanRegistrar {  
    @Override  
    public void register(BeanRegistry registry, Environment env) {  
        registry.registerBean(PetService.class);  
        if (env.matchesProfiles("dev")) {  
            registry.registerBean(TestingController.class,  
                (c) → c.supplier(context →  
                    new TestingController(context.bean(PetService.class))  
                )  
            );  
        }  
    }  
}
```



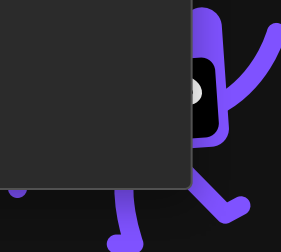
Kotlin DSLs in Spring

```
class Registrar : BeanRegistrarDsl({  
    registerBean<PetService>()  
    if (env.matchesProfiles("dev")) {  
        registerBean { TestingController(bean()) }  
    }  
})
```



Kotlin DSLs in Spring

```
registerBean {  
    val http = bean<HttpSecurity>()  
    http {  
        cors { disable() }  
        csrf { disable() }  
        authorizeHttpRequests {  
            authorize("/admin/**", hasRole("ADMIN"))  
            authorize(anyRequest, authenticated)  
        }  
        formLogin { }  
        httpBasic { }  
    }  
    http.build()  
}
```



Router DSL

```
fun petHandler(petService: PetService) = router {  
    "/pets".nest {  
        GET {  
            val pets = petService.getAllPets()  
            ok().body(pets)  
        }  
        POST {
```

Router DSL

```
class Registrar : BeanRegistrarDsl({  
    registerBean<PetService>()  
    registerBean { petHandler(bean()) }  
})
```

```
@Configuration
```

```
@EnableAutoConfiguration
```


When should you use these DSLs?

- Small to medium microservices
- startup time important
- you like functional style programming & apis
- always use test DSLs



What makes Kotlin & Spring interop comfortable?

KOTLIN

SPRING



What makes Kotlin & Spring interop comfortable?

KOTLIN

```
class PetService  
final by default
```

SPRING



What makes Kotlin & Spring interop comfortable?

KOTLIN

```
class PetService  
final by default
```

SPRING

```
CGLIB proxy  
@Transactional, @Configuration, ...
```



What makes Kotlin & Spring interop comfortable?

KOTLIN

```
class PetService  
final by default
```

cannot subclass

SPRING

```
CGLIB proxy  
@Transactional, @Configuration, ...
```



What makes Kotlin & Spring interoperable?

KOTLIN

```
class PetService  
final by default
```

SPRING

```
CGLIB proxy  
@Transactional, @Configuration, ...
```

cannot subclass

```
kotlin("plugin.spring")
```



What makes Kotlin & Spring interop comfortable?

KOTLIN

```
class PetService  
final by default
```

SPRING

```
CGLIB proxy  
@Transactional, @Configuration, ...
```

cannot subclass

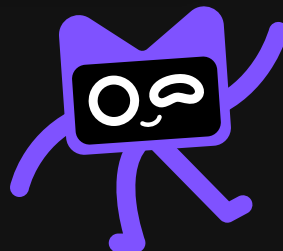
```
kotlin("plugin.spring")
```

```
open for Spring annotated classes & methods
```



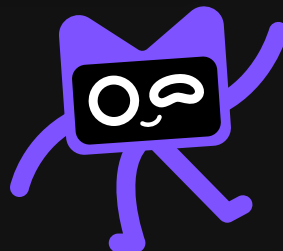
Spring Compiler Plugin

```
@Service
open class PetService {
    @Transactional
    open fun scheduleVisit(
        petId: Long,
        at: OffsetDateTime = OffsetDateTime.now() + Duration.ofDays(2)
    ): Visit {
        ...
    }
}
```



Spring Compiler Plugin

```
@Service
class PetService {
    @Transactional
    fun scheduleVisit(
        petId: Long,
        at: OffsetDateTime = OffsetDateTime.now() + Duration.ofDays(2)
    ): Visit {
        ...
    }
}
```



JPA Compiler Plugin

```
plugin {  
    kotlin("plugin.jpa")  
}
```



JPA Compiler Plugin

```
@Entity
open class Pet() {
    @Id
    open var id: Long? = null
    open var name: String? = null

    constructor(
        id: Long? = null,
        name: String? = null,
    ) : this() {
        this.id = id
        this.name = name
    }
}
```

added for proper subclassing by hibernate

NoArg constructore for Hibernate

JPA Compiler Plugin

```
@Entity
class Pet(
    @Id
    var id: Long? = null,
    var name: String,
)
```

NoArg constructore generated & class opened



Lombok?

- Kotlin cannot see Lombok's generated declarations
- Historically presented a blocker to Kotlin Adoption
- experimental Kotlin Lombok Plugin in 1.5.20
- promoted to alpha in 2.3.20
- runs lombok before `kotlinc`

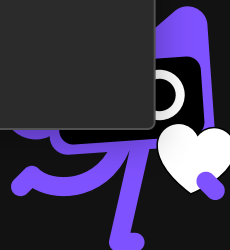
```
plugins {  
    kotlin("plugin.lombok")  
}
```



Future Improvements for Kotlin Lombok Plugin

```
@Builder
@AllArgsConstrcutor
@Setter
@Getter
class Pet {
    private Long id;
    private String name;
    private Breed race;

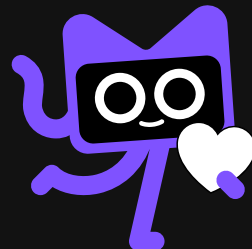
    enum Breed {
        CAT, DOG, HORSE,
    }
}
```



misses the Builder

Future Improvements for Kotlin Lombok Plugin

```
class Pet(  
    var id: Long,  
    var name: String,  
    var race: Breed,  
) {  
    enum class Breed {  
        CAT, DOG, HORSE,  
    }  
}
```

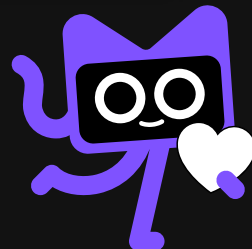


Future Improvements for Kotlin Lombok Plugin

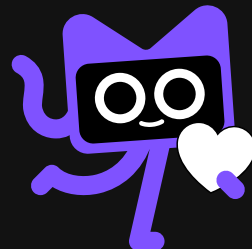
`@Builder`

Kotlin Plugin will generate Builder

```
class Pet(  
    var id: Long,  
    var name: String,  
    var race: Breed,  
) {  
    enum class Breed {  
        CAT, DOG, HORSE,  
    }  
}
```

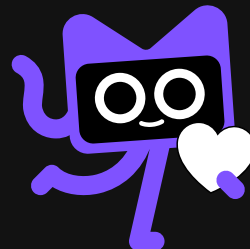


Future Improvements for Kotlin Lombok Plugin



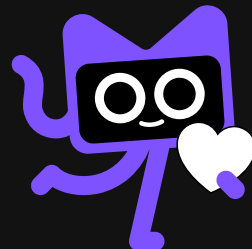
Future Improvements for Kotlin Lombok Plugin

- `@Builder`



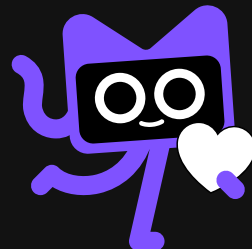
Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...



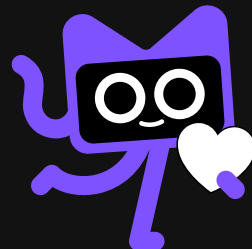
Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...
- `@NoArgsConstructor`



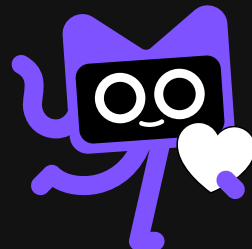
Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...
- `@NoArgsConstructor`
- `@EqualsAndHashCode`



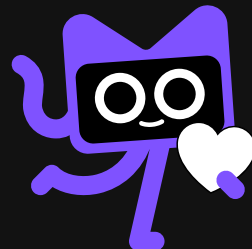
Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...
- `@NoArgsConstructor`
- `@EqualsAndHashCode`
- `@ToString`



Future Improvements for Kotlin Lombok Plugin

- `@Builder`
- `@Log` , `@Slf4j` , ...
- `@NoArgsConstructor`
- `@EqualsAndHashCode`
- `@ToString`
- will probably arrive in `2.5.x`



Kotlin Ecosystem

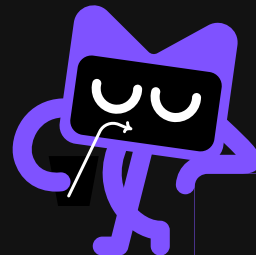
- Mappie
- MockK
- Kotest
- Exposed



Mappie

- alternative to MapStruct
- Compiler Plugin
- very good error messages

```
object PetMapper : ObjectMappie<Pet, PetDto>
```



Mappie

- alternative to MapStruct
- Compiler Plugin
- very good error messages

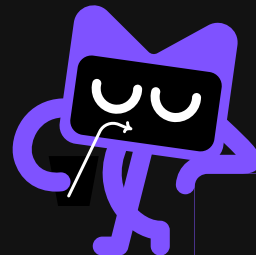
```
Target PetDto::id automatically resolved from Pet::id  
but cannot assign source type Long? to target type Long
```



Mappie

- alternative to MapStruct
- Compiler Plugin
- very good error messages

```
object PetMapper : ObjectMappie<Pet, PetDto> {  
    override fun map(from: Pet): PetDto = mapping {  
        PetDto::id fromPropertyNotNull from::id  
    }  
}
```



MockK

- excellent mocking library for Kotlin
- Kotlin DSL
- Spring Support through `SpringMockK`

```
val pet = mockk<Pet>()
every { pet.name } returns "Fido"
```



MockK

- excellent mocking library for Kotlin
- Kotlin DSL
- Spring Support through `SpringMockK`

```
@SpringBootTest
class PetControllerTest {
    @MockkBean
    private lateinit var petService: PetService

    @Test
    fun `should return pet`() {
        every { petService.getPet(1L) } returns Pet(1L, "Fido")
    }
}
```



Kotest

- Test Framework with multiple styles
- assertions
- property testing



Kotest

```
class MyTests : FunSpec({  
    test("String length should return the length of the string") {  
        "sammy".length shouldBe 5  
        "".length shouldBe 0  
    }  
})
```

Kotest

```
class MyTests : ShouldSpec({  
    should("return the length of the string") {  
        "sammy".length shouldBe 5  
        "".length shouldBe 0  
    }  
})
```


Kotest

```
class MyTests : BehaviourSpec({  
    given("a string") {  
        `when`("i measure it's length") {  
            then("it should have length of 5") {  
                "sammy".length shouldBe 5  
            }  
        }  
    }  
})
```

Kotest

```
@Test
fun `length of string`() {
    "sammy".length shouldBe 5
}
```

Exposed by JetBrains

- Kotlin DB library with ORM & query DSL flavours
- JDBC & R2DBC Support
- Generate Flyway migrations from Table definitions
- Spring Boot integration



Provides Id Column Exposed by JetBrains

```
object PetTable : LongIdTable() {  
    val name = varchar("name", 50)  
    val age = integer("age")  
    val breed = enumeration<Breed>("breed")  
}
```

Typesafe column definition

```
data class Pet(  
    val id: Long,  
    val name: String,  
    val age: Int,  
    val breed: Breed,  
)
```

built in enumeration support

Exposed by JetBrains

```
@Transactional
```

```
fun getPetById(id: Long) = PetTable.selectAll().where {  
    PetTable.id eq id typesafe comparisons  
}.firstOrNull()?.toPet()
```

```
fun ResultRow().toPet() = Pet(  
    id = this[PetTable.id],  
    name = this[PetTable.name],  
    age = this[PetTable.age],  
    breed = this[PetTable.breed],  
)
```

typesafe access to columns

Exposed by JetBrains

DAO definition

```
class PetEntity(id: EntityID<Long>) : LongEntity(id) {  
    companion object : LongEntityClass<PetEntity>(PetTable)  
    val name by PetTable.name  
    val age by PetTable.age  
    val breed by PetTable.breed  
}
```

Provides CRUD

Describes how to resolve entity field from Table

Exposed by JetBrains

```
fun createPet() {  
    PetEntity.new {  
        name = "Loki"  
        age = 10  
        breed = Breed.CAT  
    }  
}  
  
fun findPetById(petId: Long): PetEntity = PetEntity.findById(petId)
```

creates and saves new PetEntity

also provided by companion

Q & A

Slides

